Evaluation-Driven Release Governance for LLM Routing in Production: A Case Study of FrugalGPT Router Pro × Aegis

Jiarui Wang, School of Mathematics, Xi'an University of Technology Email: 3759513835@qq.com · ORCID: https://orcid.org/0009-0006-5054-6062

Abstract

Large language model (LLM) routing has become an essential layer for balancing quality and cost in production deployments, yet most published routers stop at the algorithm: they report benchmark numbers but provide no evidence-based procedure for deciding when a candidate router is safe to deploy, how to compare it against the incumbent under real traffic, or how to roll back when it regresses. This paper presents a production system, FrugalGPT Router Pro coupled with the Aegis observability and governance platform, that treats router deployment as a governed release process. The system combines a four-stage decision chain (safety, explicit rules, learned routing, and cascade) with a six-loop closed governance cycle of observation, aggregation, retraining, candidate selection, A/B testing, and adjudication. We evaluate the system through preregistered statistical protocols on a 2,000-question, five-provider benchmark and a 313-question, 14-domain paid production workload. The learned router achieves statistically significant quality gains over a lightweight baseline on three of four providers (e.g., +15.2 points on stepfun, $p = 3.7e\text{-}40$) while reducing cost by 36-51% relative to a heavyweight baseline; in the paid workload it attains a 98.1% success rate, 59.65% cost savings, a calibrated quality retention of 0.9278 (95% CI [0.892, 0.953]), and a decision-latency p99 of 0.169 seconds. We also report a negative result with practical value: after rule tuning, our learned router did not significantly outperform the best-tuned heuristic on any provider of this benchmark, indicating that hand-crafted rules already capture much of the routable signal within this classifier family and rule grid. Finally, we show that the release gate correctly blocked two candidate versions (v12 and v13) that raised quality-downgrade flags in blind evaluation, keeping the incumbent v7 online; across the full registered version history (v1-v17), all eight candidates subjected to the gate were rejected, blocked, rolled back, or left unpromoted, with zero recorded erroneous promotions. We argue that the governance loop, rather than the routing algorithm alone, is the transferable contribution for industry-track deployment.

1. Introduction

The commercial LLM ecosystem offers models that differ by an order of magnitude or more in price and capability, and no single model dominates every query (Lu et al., 2026). A growing body of work therefore treats model selection as a routing problem: given a query and a pool of models, decide which model should answer it. Early formulations such as FrugalGPT (Chen et al., 2023) proposed cascades and prompt adaptation to reduce cost while preserving quality, and subsequent work developed learned routers that train on preference data (Ong et al., 2024), adaptive routers that respect budget constraints (Panda et al., 2025), production-oriented routers with hybrid offline-online learning (Bao, 2026), ensemble routers (Wang et al., 2025), cost-aware rate-optimal routers (Somerstep et al., 2025), and unified theories of routing and cascading (Dekoninck et al., 2025).

Despite this algorithmic abundance, the operational layer around routers remains underdeveloped. Published work typically evaluates a router once, on a fixed benchmark, and reports an aggregate accuracy or cost figure. Production operators, by contrast, face a different set of questions. How do we know a retrained router is actually better than the one currently serving traffic? How do we distinguish a genuine improvement from noise when the evaluation judge itself is an LLM with known biases (Zheng

et al., 2023; Krumdick et al., 2025)? How do we prevent an aggressive candidate from silently degrading quality in a narrow domain that a single aggregate metric cannot see? And once a candidate passes, how do we roll back if it regresses under real load?

These questions matter because a router is not a static artifact. In a production environment the router is retrained on rolling traffic windows, candidate versions are generated continuously, and each promotion is a small deployment decision with real financial and quality consequences. A router that upgrades too aggressively spends money without improving answers; a router that downgrades too readily lets quality slip beneath the expectations of the applications that depend on it. Point estimates on a static benchmark cannot arbitrate these trade-offs, because they ignore the paired structure of the data, the varying difficulty of the traffic, the bias of the evaluator, and the possibility of regression in a subset of domains that an aggregate number hides.

This paper reports on a production system built to answer these questions. FrugalGPT Router Pro is an enterprise routing gateway that combines a lightweight learned router with explicit fallback and safety stages, and Aegis is the observability and governance platform that surrounds it with instrumentation, audit logging, automated retraining, and a release gate. Together they implement what we call evaluation-driven release governance: a preregistered, statistically grounded procedure that decides, on evidence, whether a candidate router may be promoted to production. The core idea is that a router release should be treated the way a well-run software organization treats a service deployment: every candidate is versioned, every claim about it is tested against a control, every disagreement between candidate and incumbent is examined on the actual generated text, and every promotion leaves an audit trail from which the decision can be recomputed. We report this experience so that other operators can adopt the protocol rather than rediscovering its components.

The contributions of this paper are threefold. First, we describe a complete closed governance loop for router deployment, from telemetry capture through candidate generation and A/B adjudication to release or rollback, and we position it against existing commercial observability and lifecycle tooling, which covers monitoring but stops short of evidence-gated release management. Second, we evaluate the system with preregistered statistical protocols, including paired McNemar tests with Holm correction, equivalence tests with a delta of three accuracy points, and a three-caliber accuracy convention that separates fair, determinable, and conservative estimates, together with a stratified human audit of the judge itself, and we report both positive and negative empirical findings from real paid API traffic. Third, we document a release-gate case study in which two candidate versions that looked excellent on decision accuracy were blocked by blind review of the text they actually generated, together with the disclosure discipline (including our own historical gate bypass) that the audit trail requires. We argue that the governance protocol, not the routing algorithm, is the transferable contribution.

The remainder of the paper is organized as follows. Section 2 reviews related work on routing algorithms, LLM-based evaluation, and production observability. Section 3 describes the system architecture and the governance loop. Section 4 specifies the evaluation methodology, including data, protocols, preregistered criteria, and released artifacts. Section 5 presents the results. Section 6 discusses implications and limitations, and Section 7 concludes.

2. Background and Related Work

2.1 LLM Routing

The routing literature can be organized along two axes: what signal the router uses and how the router is trained. FrugalGPT (Chen et al., 2023) introduced cost-aware cascading as a practical way to reduce inference cost while maintaining answer quality, establishing the economic motivation that later routers inherit. RouteLLM (Ong et al., 2024) framed routing as a learning problem over preference data, showing that lightweight rankers trained on user preferences can approach the quality of much larger models at a fraction of the cost. Panda et al. (2025) studied routing under explicit budget constraints and showed that an adaptive router can remain within budget while preserving utility. OrcaRouter (Bao, 2026) targets production settings directly, combining offline pretraining with online bandit-style updates to adapt to traffic drift. MixLLM (Wang et al., 2025) routes across an ensemble of open and commercial models, and CARROT (Somerstep et al., 2025) derives rate-optimal policies under cost awareness. On the theoretical side, Dekoninck et al. (2025) unify routing and cascading, giving conditions under which deterministic routing decisions are optimal and connecting router design to classical prediction with expert advice.

Two recent lines are especially close to our operational setting. IPR (Feng et al., 2025) routes queries to a portfolio of LLMs under a user-controlled quality-cost trade-off parameter and reports a 43.9% cost reduction in a production deployment, with a tolerance parameter to guard against quality violation; it demonstrates that industrial operators are already running router-style systems as governed services, although its evaluation centers on cost-quality trade-offs rather than a release protocol. UniRoute (Jitkrittum et al., 2025) and PORT (Wu & Silwal, 2025) pursue training-free online routing with theoretical competitive-ratio guarantees at zero training overhead; PORT in particular frames routing as an online decision problem with worst-case guarantees, which complements rather than replaces the empirical release protocol we describe.

Alongside individual algorithms, the community has begun to build standard evaluation infrastructure. RouterArena (Lu et al., 2026) is an open platform for comparing LLM routers, providing a knowledge-covering dataset, per-domain difficulty levels, a broad set of metrics, and an automated leaderboard update framework; RouterBench (Hu et al., 2024) complements it with a unified router interface and a 405k-inference-result dataset designed for standardized router evaluation; and RouterEval (Huang et al., 2025) extends the line to large scale, building a routing benchmark from over 200 million inference records spanning more than 8,500 LLMs. At the same time, at least one workshop study reports practical lessons from applying LLM routing across varied task settings (Srivatsa et al., 2024), which we read as consistent with the scope-dependent negative result we report in Section 5.2. The existence of such platforms is itself a sign of how crowded the space has become: new routers appear faster than the community can vet them, which makes the question of how a given router should be operated in production all the more pressing.

What the algorithm literature has in common is that evaluation is a one-shot activity on a static benchmark, usually with a single accuracy or cost number per router. None of these works describes a production procedure for deciding, on an ongoing basis, whether a newly trained router may replace the one in service, how to detect that it has silently regressed, or how to reverse the decision. The system we describe in this paper does not aim to beat these algorithms on a leaderboard; it aims to make their deployment governable, and it treats the router as a continuously updated service whose releases require evidence.

2.2 LLM-Based Evaluation

Because production routers must be evaluated continuously on heterogeneous traffic, and because human labels are expensive, the field has turned to LLMs as judges. Zheng et al. (2023) showed that strong LLMs can agree with human preferences at levels comparable to inter-human agreement on MT-Bench and Chatbot Arena, while also documenting position bias and verbosity bias. G-Eval (Liu et al., 2023) demonstrated that chain-of-thought scoring with GPT-4 achieves high correlation with human judgments. Recent surveys consolidate this literature and catalog judge biases systematically (Gu et al., 2024), and JudgeLM (Zhu et al., 2025) adds a systematic analysis of position, knowledge, and format biases together with a fine-tuned judge that mitigates them. Self-preference bias, in which a judge favors outputs that are more familiar to it (e.g., those with lower perplexity), is a further documented mechanism (Wataoka et al., 2024). More recently, Krumdick et al. (2025) examined the limits of LLM-as-a-judge without human grounding and showed that judge-based rankings can diverge from human-labeled ground truth in systematic ways, and Meta-Router (Zhang et al., 2025) proposes to bridge preference-based and gold-standard evaluations in the routing setting.

Two protocol-level findings shaped our judge design. Tripathi et al. (2025) show that pairwise feedback protocols are substantially more vulnerable than absolute scoring to distractor manipulation (preferences flipped in about 35% of manipulated comparisons versus 9% for absolute scores), which supports our choice of calibrated pointwise scores as the primary production signal; Jung et al. (2025) prove human-agreement guarantees for abstaining when an LLM judge is not sufficiently confident, the mechanism our blind-review stage approximates by escalating low-confidence or disagreement samples to finer-grained adjudication.

Our experience is consistent with this literature and adds a concrete engineering lesson: in a production routing context, the judge's errors are not uniform. We observed a +13.3-point systematic over-estimation on the difficult segment of a benchmark, replicated across two independent experiments, which means a single global judge threshold will systematically miss upgrade needs for hard queries. We therefore calibrate judge scores per difficulty segment rather than globally, and we treat the judge as a measured instrument whose bias is monitored and disclosed rather than assumed neutral. This is a prerequisite for trusting any quality comparison between a candidate and an incumbent, because a biased judge can certify a worse router with high apparent confidence.

2.3 Production Observability and Release Management

Commercial tooling for LLM operations has grown quickly: observability platforms such as Langfuse and LangSmith provide tracing and evaluation dashboards, and gateway products such as LiteLLM and Portkey offer model routing and key management. On the research side, the WRP architecture (Chen et al., 2026) surveys the workload-router-pool design space for LLM inference and catalogs governance concerns such as policy conflicts and fleet-level routing adaptation; its emphasis is capacity planning and energy, and it does not specify an evidence protocol for promoting or rolling back a router version, which is the gap this paper addresses. On the lifecycle side, Generative AI Toolkit (Kohl et al., 2024) proposes a DevOps framework covering development, deployment, and operations of LLM applications, and IPR (Feng et al., 2025) is deployed as a production routing service with continuous operation. These systems cover monitoring and operation well, but to our knowledge none provides the three release-management capabilities we combine as a single integrated loop: an automatic retraining trigger with an evidence gate, router strategy versioning with rollback, and a preregistered A/B adjudication protocol in which disagreements between candidate and incumbent are adjudicated on generated text rather than on the router's own labels. That gap is precisely the niche our system occupies, and it is the aspect of the work

that is most directly relevant to industry-track and systems venues. The systems community has long understood that the hard part of operating a learned component is not training it once but shipping it repeatedly without regression; this paper applies that discipline to LLM routers specifically.

Table 1 positions the system against the three families of prior work along the operational capabilities that matter for governed deployment. Algorithm-oriented routing papers (RouteLLM; IPR; UniRoute; PORT; MixLLM; CARROT) contribute routing policies but, with the partial exception of IPR's tolerance parameter, do not specify an evidence procedure for promoting or rolling back a deployed version. Evaluation platforms (RouterArena; RouterBench) standardize offline comparison but deliberately stop at a leaderboard number. Observability and gateway tooling (Langfuse; LangSmith; LiteLLM; Portkey) covers tracing and evaluation dashboards but no release-gate semantics. The combination of all eight capabilities in a single, auditable loop is, to our knowledge, absent from the published literature. A quantitative head-to-head against observability or gateway tooling is deliberately not attempted: Langfuse and LangSmith operate at the tracing and evaluation-dashboard layer, LiteLLM and Portkey at the request-gateway layer, whereas the release gate is a deployment-decision layer above all of them. The three layers answer different questions, so a single accuracy or latency number would not be a meaningful comparison; the transferable claim of this paper is the gate protocol itself, not a speed or quality margin over a gateway.

Capability	Routing algorithms	Evaluation platforms	Observability/gateway tooling	This system
Versioned router artifacts	Partial (IPR as a service; most are static)	—	Partial (gateway configs)	Yes
Rolling retraining trigger	Partial (IPR hybrid online updates)	—	—	Yes (traffic windows + explicit retraining)
Preregistered statistical protocol	—	—	Partial (eval dashboards)	Yes (J1-J4, paired McNemar, equivalence)
Evidence-gated release decision	Partial (IPR tolerance parameter)	—	—	Yes (explicit numeric gate)
Blind adjudication on generated text	—	—	—	Yes (stratified disagreement review)
Rollback path	—	—	Partial	Yes (versioned backups; v4 case)
Versioned, disclosed thresholds	—	—	—	Yes (2026-10-11 revision disclosed)
Judge bias measurement & calibration	—	—	Partial (Langfuse evals)	Yes (per-difficulty calibration)
End-to-end recomputable audit	Partial (IPR)	Partial	Partial	Yes (audit + billing ledger, hashed)

3. System Design

3.1 FrugalGPT Router Pro: The Gateway

FrugalGPT Router Pro is a stateless HTTP gateway that exposes an OpenAI-compatible chat completions endpoint. For each request it runs a four-stage decision chain. The first stage is a safety and policy layer provided by Aegis: content-safety classification, PII detection, and injection guards run before any model is contacted; flagged requests are rejected with an explicit error or force-routed to the flagship tier,

depending on the policy class, so that the routing decision is never made on disallowed content. The second stage applies explicit deterministic rules, such as length and digit-density heuristics and explicit routing overrides; these rules are cheap, auditable, and intentionally placed before the learned component so that well-understood patterns never depend on a statistical model. The third stage is a learned router: a TF-IDF feature representation over the query text (character-level n-grams) fed into a logistic regression classifier that produces a confidence score; a threshold theta (by default 0.4) maps the score to a tier. The fourth stage is a cascade fallback across providers with a health pool: if the selected provider fails, the gateway degrades to the next healthy provider in the tier, and circuit breakers trip on repeated 401, 402, 429, or quota errors without stalling the request.

The tier model is deliberately coarse. Each provider exposes two or three tiers, conventionally labeled light, medium, and heavy, with different prices per million tokens. The router's job is to place each request at the cheapest tier that can answer it, subject to the learned confidence and the safety net constraints described below. Keeping the decision surface coarse has a practical advantage: it makes the router's behavior legible to operators, it bounds the number of configuration knobs, and it makes A/B comparisons between versions meaningful, because two versions differ only in where they draw the tier boundary.

3.2 Aegis: Observation and Governance

Aegis is the observability and governance layer that surrounds the gateway. Every request produces an audit record with decision, tier, provider, token counts, cost, latency, and quality score, and every response produces a billing line in a ledger that can be replayed for exact accounting. A local judge model (Qwen3-8B) scores responses immediately, and the judge's outputs are themselves monitored for drift through difficulty-ordered online streams that alert when a segment's score distribution shifts. All artifacts are versioned and hashed so that any reported number can be recomputed from raw evidence; this is what makes the evaluation protocol auditable by a third party rather than self-reported.

3.3 The Six-Loop Governance Cycle

The distinguishing feature of the system is the closed governance loop that surrounds the router. The loop has six stages (Figure 1). Observation collects audit and quality signals from live traffic. Aggregation rolls the signals into windows with defined time ranges and filters. Retraining consumes aggregated windows to produce candidate router versions; the system supports both explicit retraining commands and an online learning path that auto-harvests traffic during a run. Selection applies an evaluation gate that scores each candidate against the incumbent on preregistered criteria. A/B testing splits a fraction of live traffic between the candidate and the incumbent and measures both quality and cost under real load. Adjudication combines the offline gate, the A/B measurements, and a blind evaluation on disagreements, and either promotes the candidate or keeps the incumbent.

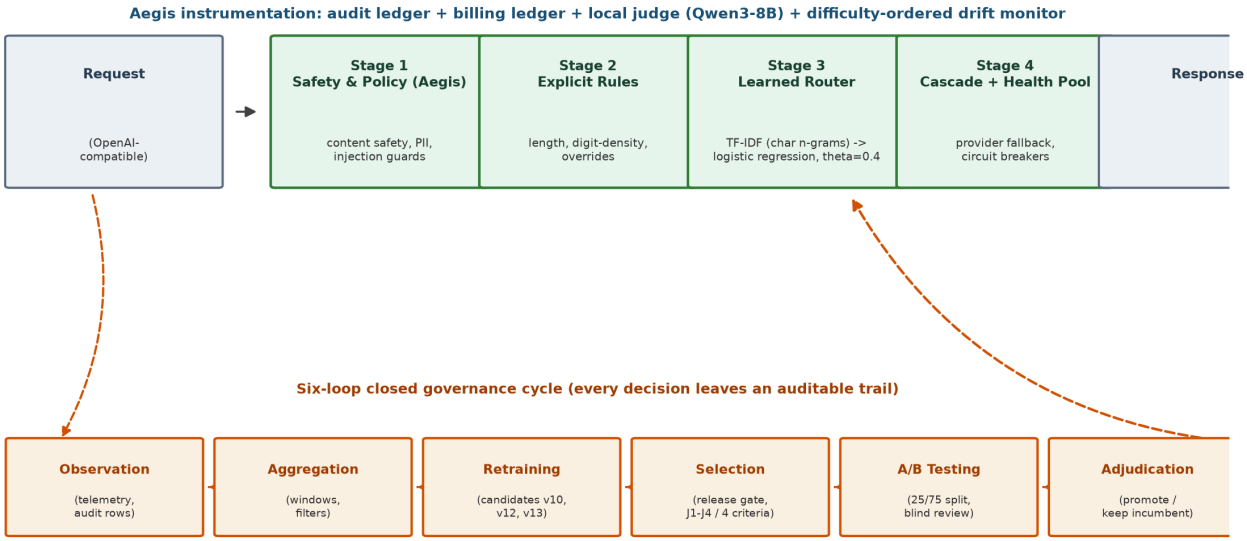


Figure 1. System architecture: the four-stage routing gateway (safety, explicit rules, learned router, cascade) instrumented by Aegis, and the six-loop closed governance cycle of observation, aggregation, retraining, selection, A/B testing, and adjudication.

The release gate is defined by explicit numeric criteria, and the thresholds themselves are versioned and revised on evidence. The initial gate, which was in effect when the benchmark evaluation was preregistered and when the production workload was run, required a candidate to pass all four absolute criteria: quality retention at least 0.95 relative to the incumbent, cost savings above a floor of 0.50, decision latency p99 within 2.0 seconds, and a judged sample size above 30. Operational experience then exposed a calibration problem: on real business traffic the control arm's own quality pass rate was only about 69.7%, so an absolute quality floor of 0.95 was almost unreachable for any candidate and made the gate effectively vacuous in the wrong direction. On 2026-10-11, before the v12/v13 adjudication window was closed, the gate was therefore revised on evidence: live A/B adjudication moved to a relative-offset criterion (candidate quality at least the control's quality minus one percentage point, and candidate cost saving at least the control's saving minus two points), retained an absolute safety net of 0.50 on cost saving, and lowered the offline gate's quality floor from 0.95 to 0.90, after an authoritative re-measurement of the incumbent v7 came in at 92.78% quality retention. This revision is disclosed here because a governance claim is only as credible as its threshold history. We also note, for readers who might read the revision as moving the goalposts, that the incumbent v7 had already passed the original absolute thresholds in a gate re-check on live traffic before the revision (calibrated quality retention 0.9857 against the 0.95 floor, among the other three criteria); the revision was motivated by the unreachability of the absolute floor for candidates on real traffic, not by any need to admit the incumbent. The gate is deliberately conservative: failing is the expected outcome for most candidates, and the failure record is treated as evidence that the process is working, not as a defect in the system. In the language of deployment practice, the gate is a quality gate in the release pipeline: it exists to make the no-decision the default, and to require positive evidence before a candidate displaces a working incumbent.

4. Evaluation Methodology

4.1 Benchmark Data

The primary benchmark, referred to as bench2000, consists of 2,000 questions drawn from GSM8K, MMLU, and locally authored arithmetic and multiple-choice items, assembled with a fixed random seed of 42. The set is stratified across nine question-type groups: 500 GSM8K arithmetic word problems, 500 MMLU-style multiple-choice items with options indexed 0-3, 220 dense-digit problems, 200 long-context arithmetic problems, 200 long-context multiple-choice questions, 100 mixed-difficulty arithmetic, 100 mixed-difficulty multiple choice, 100 adaptive mixed-pool questions, and 80 elementary multiple-choice items with numeric options. The full manifest, including per-group item counts, source attribution, and the fixed seed, is released with the paper (see Section 4.5). The evaluation pool spans five providers with light and heavy tiers, including deepseek, zhipu, silicon, stepfun, and gitee; gitee exposes no heavy tier and is therefore analyzed only on the light side. All tiers run with a unified max_tokens budget of 1,024, and all costs are computed from actual API usage fields, including reasoning tokens, using published platform prices.

Truncation is a first-class concern in this benchmark. The heavyweight reasoning models occasionally exceed the token budget on long questions, and when they do, the final answer field is empty while the reasoning trace holds the answer. The fair caliber therefore excludes truncated requests; the conservative caliber counts them as wrong. The observed truncation rates are documented for every arm, and the highest is 33.1% on the stepfun heavy tier, which is also the provider whose light-to-heavy accuracy gap is the largest by far, a combination that illustrates why the conservative caliber is reported alongside the fair one.

Three accuracy calibers are reported. The fair caliber (V) excludes requests whose completion hit the token limit, comparing arms on their overlapping determinable subset. The determinable caliber (S) considers all requests with a determinable answer. The conservative caliber (F) counts truncated requests as wrong. Because the four arms have different effective sample sizes after per-arm truncation exclusion, cross-arm accuracy is only approximately comparable; all significance claims are based on paired tests over the intersection of the two arms, which is statistically valid.

4.2 Preregistered Criteria and Statistical Protocol

Before running the benchmark, the evaluation protocol fixed a set of criteria, labeled J1 through J4. J1 requires the rule baseline to exceed the lightweight baseline by at least two points, establishing that a routing signal exists. J2 requires the learned router to be significantly better than both the rule baseline and the lightweight baseline after Holm correction, in the direction claimed. J3 requires the learned router to keep its arm-level accuracy within three points of the heavyweight baseline (equivalence with a delta of three accuracy points, computed on the arm-level aggregate accuracy after per-arm truncation exclusion). J4 requires the learned router's cost to be no more than half of the heavyweight tier's cost while remaining at least twice the lightweight tier's cost, so that the saving is real but not implausibly large. The criteria were fixed before the numbers were computed, which prevents the protocol from being redefined in light of the results.

Paired comparisons use the exact binomial version of McNemar's test, where the statistic counts the number of items on which the two arms disagree and the test is a two-sided exact binomial test. The implementation was validated against known values before use (for example, a Bin(10, 0.5) tail of at most 1 reproduces exactly the textbook probability). All p-values are reported alongside the direction of the difference, and Holm correction is applied for multiple comparisons across the criteria so that the family-

wise error rate remains controlled when several tests are evaluated together. Judge quality was audited by stratified sampling: 36 samples were independently reviewed by a human, with agreement on 35 of 36 samples (97.2%, Wilson 95% CI [85.8%, 99.5%]); the single disagreement was traced to a LaTeX thousands-separator parsing bug that was subsequently fixed and folded into the frozen judge version, and a five-item correctness reversal was confirmed as a result of that fix.

4.3 Production Workload

In addition to the offline benchmark, the system was evaluated on a paid production workload of 313 questions across 14 professional domains (general, code, multilingual, legal, marketing, research, extreme, math, logic, finance, medical, agent, knowledge, and safety). The routing arm sent all 313 questions to the live production endpoint at port 9011 (engine v1.17.6, online router v7, default threshold theta = 0.4) at temperature zero and with no mocks, so that every decision was made by the exact artifact that serves production traffic; the control arm sent the same questions in the same order to a direct heavyweight provider call (deepseek-v4-pro), so that the cost comparison is a same-load, same-order measurement rather than an extrapolation.

Audit and billing records were persisted for the full window, and a local judge scored each response in real time; quality retention is judge-calibrated as described in Section 5.5. For the confidence interval reported in Section 5.3, we treat all 313 workload responses as the evaluation denominator (the six non-successful requests counted as non-retained) and use the exact Clopper-Pearson interval; the interval reflects sampling variability of the underlying retention rate, not calibration bias. A separate external blind evaluation scored the same workload family with a different judge (DeepSeek-V3.1, anonymous presentation, temperature 0) to provide an independent cross-check on a different question set and sampling; the two quality figures are explicitly treated as cross-checks, not as interchangeable estimates of the same quantity.

4.4 A/B Testing Protocol

Candidate versions were evaluated with a two-stage protocol. First, a blind comparison generated both versions' answers on the same questions and sampled 20% of disagreements, stratified by upgrade and downgrade direction with a fixed seed, for judge review; the blind step is essential because it examines actual generated text rather than the router's own labels. Second, a live A/B test routed 25% of fresh business traffic to the candidate and 75% to the incumbent for a defined window, and the two arms were compared on quality pass rate, cost savings, and decision accuracy. The adjudication combined the offline gate, the blind disagreement review, and the live A/B statistics; any evidence of quality downgrade on upgraded requests triggered a manual block per the methodology, regardless of what the aggregate A/B statistics happened to say. Algorithm 1 summarizes the complete adjudication as a single procedure.

Algorithm 1: Release-gate adjudication for a candidate router Input: candidate c, incumbent b, preregistered criteria J1-J4, gate thresholds G = (q_min, s_min, t_p99, n_min), blind-review sampling policy, A/B split ratio r. **Output:** promote or block, together with a versioned, hashed evidence record.

1. Offline gate: run c and b on the bench2000 benchmark; compute paired McNemar statistics and the preregistered criteria J1-J4.
2. **if** any preregistered criterion fails **then** record the reason; **return** block.

3. Threshold gate: require quality retention q(c,b) >= q_min, cost saving s(c,b) >= s_min, decision latency p99 <= t_p99, and judged sample size >= n_min; **if** any fails **then** record the reason; **return** block.
4. Blind review: sample disagreements stratified by upgrade and downgrade direction with a fixed seed; adjudicate the generated text; **if** upgrades show a quality downgrade **then** issue a manual block.
5. Live A/B: split r / (1-r) of fresh traffic for a defined window; compare quality pass rate, cost saving, and decision accuracy under relative-offset criteria with an absolute safety net.
6. Adjudicate: combine steps 2, 3, 4, and 5; **if** any downgrade evidence stands **then** block, **else** promote.
7. Audit: append the decision, the evidence files, and their hashes to the Aegis ledger; update the version register.

4.5 Artifacts and Reproducibility

The following artifacts accompany this paper. (1) The bench2000 manifest and re-check tooling: `bench2000_manifest.json` (question sources, per-group composition, fixed seed), `bench2000_results.json`, `bench2000_recheck_results.json`, and the re-check script `tools/bench2000_recheck.py`, which reproduces the 36-51% saving range and the conservative-caliber accuracy gains reported in Section 5.1. (2) The evaluation-gate tooling and evidence records used in Section 5.4, including the version evidence files for the v12/v13 A/B window and the gate re-check record for the incumbent v7. (3) A runnable community edition of the routing gateway (`frugalgpt-router`, rule-only demonstration, released under the FrugalGPT Community License 1.0 and installable via pip), which reproduces the gateway interface without the commercial learned-routing components. (4) The rule-tuning comparison data used in Section 5.2, the stratified judge-audit records used in Sections 4.2 and 5.5, and the production workload's audit and billing exports used in Section 5.3. The full artifact bundle will be made publicly available at https://github.com/Jerry-small-lucky/frugalgpt-router. All numbers in this paper can be recomputed from the corresponding audit and billing records, which are versioned and hashed in the Aegis ledger.

Reproduction walkthrough. On a machine with Python 3.11+ and access to the five providers' APIs, the production-workload result reproduces as follows. (1) Build the question set: regenerate `ultimate_prompt_set.jsonl` (313 questions, 14 domains) with the released builder script. (2) Run the routing arm: send all questions to the live endpoint `POST http://127.0.0.1:9011/v1/chat/completions` with the released engine and the v7 router artifact; the run is resumable, and each decision appends a row to `audit.jsonl` and a line to `billing.jsonl`. (3) Run the control arm: issue the same questions in the same order as direct heavyweight calls to the reference provider, so that the cost comparison is same-load and same-order. (4) Evaluate the gate: run `tools/eval_router.py --audit --window 600` to reproduce the window-600 gate report (`eval_router_report_window600.json`), and aggregate the ledger with the released aggregation script to reproduce the cost and quality figures in Table 4. Expected outputs are `run_summary.json` (success rates, tier and provider distributions, cost), the window-600 gate report, and the audit/billing exports; every figure in Section 5.3 recomputes from these files. In our environment the campaign took approximately five hours of wall-clock time and consumed a two-arm total of ¥4.89; the cost is dominated by the control arm and scales linearly with the question count.

4.6 Replication Protocol for a Second Deployment

The evidence in this paper comes from a single deployment (Section 6.3), and the single most informative next experiment is a second deployment at an independent operator. To make that test concrete and falsifiable, we specify the protocol a second operator would run; the requirements are deliberately minimal so that participation is cheap. (1) Data: one paid traffic window of at least 300 requests across at least five domains, persisted with the same audit (`audit.jsonl`) and billing (`billing.jsonl`) schemas plus a versioned router register. (2) Candidate generation: one retrained candidate produced from a rolling traffic window, plus one deliberately aggressive candidate (e.g., trained with a lower confidence threshold) to exercise the upgrade-bias failure mode that the blind review is meant to catch. (3) Procedure: run the gate exactly as in Algorithm 1, with two operator-chosen thresholds recorded before the window opens, and with the blind disagreement review and the live A/B split (25/75) executed by the operator's own staff. (4) Acceptance criteria: the protocol is judged transferable if (a) each stage blocks at least one candidate that the other stages would have released (the stage-level behavior of Table 5), (b) the blind review catches the deliberately aggressive candidate, and (c) every decision is recomputable from the operator's audit and billing records alone. (5) Reporting: the operator publishes the version register, the gate decisions, and the threshold-revision log (if any), so that the stage-level ablation and the threshold-revision discipline of Section 3.3 can be compared across environments. We have not yet executed this protocol at a second operator; this section specifies the experiment, and it is the first item on the follow-up agenda (Section 7).

5. Results

5.1 Benchmark: Learned Routing Versus Baselines

Table 2 summarizes the fair-caliber (V) results of the 2,000-question benchmark across the four providers with both tiers. The reported deltas are arm-level accuracy differences; all p values are from paired exact binomial McNemar tests on the intersection of the two arms. The learned router is significantly better than the lightweight baseline on three of the four providers: deepseek +4.0 points (p = 3.7e-8), silicon +3.7 points (p = 2.2e-8), and stepfun +15.2 points (p = 3.7e-40), while zhipu shows no significant difference (p = 0.134). Against the rule baseline, the learned router is significantly better only on stepfun (+4.2 points, p = 5.4e-5), and is not significantly different on the other three providers.

Table 2

Fair-Caliber Accuracy and Cost by Provider and Router Arm

Provider	Light acc (%)	Heavy acc (%)	Rule acc (%)	Learned acc (%)	Learned vs light (p)	Learned vs rule (p)	Learned/heavy cost saving (%)
deepseek	89.82	97.48	95.15	93.83	3.7e-8	1.000	34.5
zhipu	96.23	96.96	96.33	96.96	0.134	1.000	51.1
silicon	80.81	88.08	83.44	84.47	2.2e-8	0.319	47.5
stepfun	61.77	98.19	72.68	76.99	3.7e-40	5.4e-5	51.1

Note.

Acc = accuracy on the fair caliber (truncated requests excluded per arm). p values are from the exact binomial McNemar test. Cost saving is computed relative to the heavyweight arm using actual billed tokens. gitee is excluded because it exposes no heavy tier.

The cost results are consistent across providers. The learned router achieves cost savings between 34.5% and 51.1% relative to the heavyweight arm, and the savings were confirmed by an independent rerun with a higher token budget on the truncated heavy requests: the saving range was confirmed at 36-51% after the rerun, with fair-caliber accuracy essentially unchanged (within 0.35 points) and the conservative-caliber accuracy of the heavy tiers improving substantially (e.g., stepfun heavy +12.29 points) as truncation was eliminated.

The preregistered criteria were not fully passed by any provider: all five providers failed at least one of J1-J4 under all three calibers. The failures are informative. deepseek failed J2 (no significant gain over the rule baseline, p = 1.000), J3 (arm-level quality retention of 3.65 points, above the 3-point tolerance), and J4 (cost of ¥8.17 above half of the heavy tier's ¥12.47); zhipu failed J1 and J2 because its rule baseline barely exceeded its light tier; silicon failed J2, J3 (3.61 points), and J4 (cost of ¥1.80 above half of the heavy tier's ¥1.71); stepfun failed J3 because its heavy tier is exceptionally strong and the learned router retained only 78.4% of it; and gitee could not be evaluated on the heavy-dependent criteria. The verdict is therefore partial evidence in favor of the learned router, not a full pass, and the external narrative is kept conservative accordingly.

One convention note is worth recording: under the paired common-item convention, deepseek (−1.51 points, 90% CI [−2.12, −0.89]) and zhipu fall inside the 3-point equivalence band, while silicon and stepfun remain outside it; the arm-level convention is the binding one for the preregistered verdict because it was fixed in the protocol, and we report both rather than silently switching conventions.

A re-confirmation run is worth describing because it tests the stability of these conclusions. In it, the 1,271 heavy-tier requests that had previously been truncated were re-executed against the real five-provider APIs with a doubled token budget. Of these, 1,223 succeeded (96.2%) within the doubled budget, and the remaining 48 fell back to the authoritative cache; the incremental cost was about ¥35.5. Under the conservative caliber, the heavy tiers improved substantially, with stepfun heavy gaining 12.29 points, zhipu 5.05, and deepseek 4.60; under the fair caliber, accuracy was flat within 0.35 points. Crucially, the preregistered criteria and the McNemar conclusions were unchanged: the learned router remained significant against light on three of four providers, and all providers remained not-all-pass on J1-J4. The re-run therefore confirms both the positive and the negative findings rather than overturning them, and it pins the external saving range at 36-51%. The per-request residual truncation counts under the conservative caliber are recorded in the released re-check results file.

5.2 The Rule-Tuning Negative Result

A follow-up experiment replayed the full cached response corpus with 12 rule variants obtained by crossing length thresholds (300, 450, 600, 800) with digit-density thresholds (15, 25, 35), using the same frozen judge and the same cost model. For each provider, the best-accuracy rule variant was compared against the learned router. Table 3 reports the comparison.

Table 3

Best-Tuned Rule Versus Learned Router on Quality and Cost

Provider	Best rule (L, D)	Rule acc (%)	Learned acc (%)	Delta (learned-rule)	McNemar b/c, p
deepseek	300, 15	96.01	93.83	-2.18	18/27, p = 0.233
zhipu	300, 25	96.85	96.96	+0.11	7/15, p = 0.134
silicon	300, 15	85.19	84.47	-0.72	56/75, p = 0.115
stepfun	300, 15	83.14	76.99	-6.15	54/106, p = 4.8e-5

Provider	Best rule (L, D)	Rule acc (%)	Learned acc (%)	Delta (learned-rule)	McNemar b/c, p
gitee	300, 35	88.34	84.00	-4.34	0/0 (no information)

Note.

L = length threshold in characters; D = digit-density threshold. Rule variants were evaluated offline on cached responses with the same frozen judge and cost model. A negative delta means the tuned rule outperforms the learned router.

After tuning, our learned router does not significantly outperform the best rule on any provider, and on stepfun the best rule significantly beats it by 6.15 points (p = 4.8e-5). This claim is bounded by the scope of the experiment: it holds for this benchmark, this classifier family (TF-IDF plus logistic regression), and this 12-variant rule grid; it does not claim that learned routing is useless in general. On cost, the learned router is cheaper than the best rule variant on the four providers whose arms are directly comparable (e.g., stepfun ¥9.89 vs ¥12.60 and deepseek ¥8.17 vs ¥10.01); gitee's arms draw on different routed subsets and are not compared on cost. This negative result reframes the learned router's value: it is not a quality advantage over a well-tuned heuristic within this scope, but calibration-free operation (the optimal rule threshold differs by provider, and the default rule threshold is not optimal anywhere) plus controlled, predictable cost. The design therefore adopted a dual-engine strategy in which tuned rules are the first priority when provider differences are known and the learned router serves as the calibration-free default and the carrier of retraining gains.

5.3 Production Workload Results

Table 4 summarizes the paid production workload of 313 questions across 14 domains.

Table 4

Production Workload Results (313 Questions, 14 Domains)

Metric	Routing arm	Control (all heavy)
Success rate	98.1% (307/313)	100% (313/313)
Total cost	$0.195 (¥1.405)	$0.484 (¥3.482)
Cost saving vs control	59.65%	—
Quality retention (local judge, calibrated)	0.9278 (95% CI [0.892, 0.953])	—
External blind quality (DeepSeek-V3.1, fine-grained 60-question adjudication)	90.0% (54/60)	81.7% (49/60)
Decision latency p99	0.169 s	—
Tier distribution	Light 206 / Medium 40 / Heavy 61	—

Note.

Control arm sent the same questions to the deepseek-v4-pro heavy tier. The two quality figures use different judges and different question sets and are not directly comparable; they are reported as cross-checks on each other. The local-judge CI is an exact Clopper-Pearson interval computed on the n = 313 workload responses (six non-successful requests counted as non-retained); because the point estimate is judge-calibrated, the interval does not capture calibration bias. The exact judge-scored count and per-domain breakdown are in the released evaluation logs. An earlier exploratory external-blind run measured 83.3% (routing) versus 91.7% (flagship); it was superseded after a truncation fix and judge-

noise adjustment in the final calibration run reported here. Decision latency excludes upstream generation; component-level decision latency in microbenchmarks is far lower (P95 well under 5 ms).

Figure 5 breaks down the workload composition and the routed traffic profile. The 313 questions span 14 domains with a long-tailed distribution: general (60), code (32), extreme (30), math (26), legal (25), marketing (24), multilingual (23), safety (16), knowledge (15), logic (14), finance (13), agent (13), research (12), and medical (10); the difficulty mixture is easy 127 / medium 84 / hard 102. The routed tier distribution (light 206 / medium 40 / heavy 61) differs materially from the question set's expected-tier labels (light 127 / medium 84 / heavy 102): the v7 router conservatively placed a large share of queries in the light tier and used the medium tier far less than the set's construction assumed. Provider hits were gitee 131, siliconflow 126, and zhipu 48 (stepfun 0, because v7 did not route any request to it in this window), and the six non-successful requests comprised four safety-layer 403 rejections and two upstream timeouts. This profile matters for the governance claim in two ways. First, the routed traffic is not the benchmark: it is long-tailed over domains and heavily light-weighted, so a gate tuned on benchmark aggregates cannot be assumed to transfer without re-measurement (Section 5.7). Second, the gap between expected and routed tier distributions is exactly the kind of divergence that decision-accuracy labels can hide, a failure mode that the v12/v13 blind review later exploited (Section 5.4).

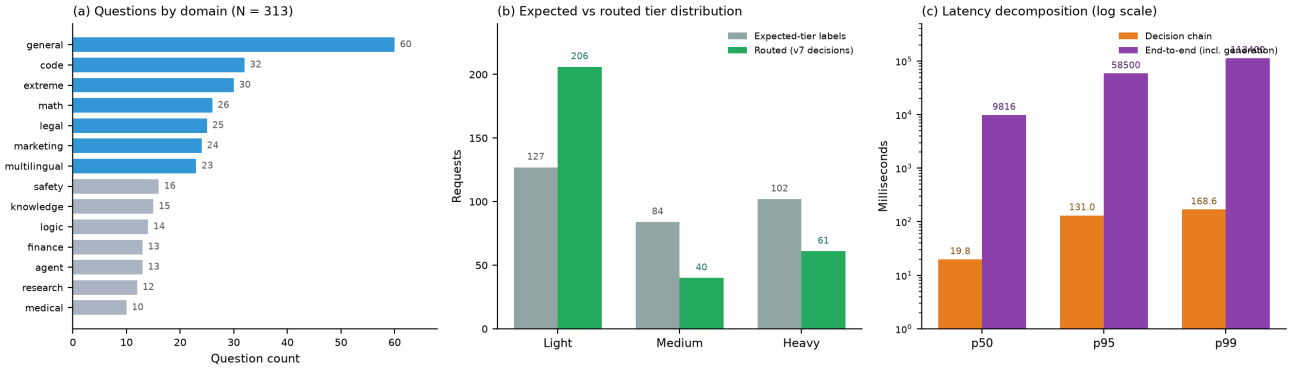


Figure 5. Production workload profile: (left) question count by domain across the 14 domains; (center) expected-tier labels versus v7's routed tier decisions; (right) decision latency (p50/p95/p99) versus end-to-end latency including upstream generation, on a logarithmic axis.

The router served 307 of 313 requests successfully, placing most requests in the light tier (206) while routing long or complex reasoning to the heavy tier (61). Of the six non-successful requests, four were correctly rejected by the safety layer with HTTP 403 for violence-related content (the safety stage's intended behavior, confirmed against the safety gateway's restricted-content grade), and two failed on upstream timeouts; all six were absorbed without interrupting service. The calibrated quality retention measured by the local judge was 0.9278 (95% CI [0.892, 0.953], exact Clopper-Pearson interval). The external blind evaluation, scored by a different judge (DeepSeek-V3.1, anonymous presentation, temperature 0) on the same workload family, measured the routing arm at 90.0% usable-quality retention (54/60) versus 81.7% (49/60) for the direct-flagship control in the final calibration run, consistent with the local estimate (0.9278) from below and resolving the tension that an earlier exploratory run (83.3% versus 91.7%) had left open. Decision latency, which excludes upstream generation, had a p99 of 0.169 seconds, far below the two-second gate. The combined two-arm cost of the workload was ¥4.89.

Beyond the headline numbers, the workload exercise demonstrated several operational properties that a routing system must have before it can be governed. The full set of 313 questions was run within a hard budget without triggering any cost stop, which shows that paid full-scale evaluation is economically

feasible at this scale. The audit subsystem persisted 621 audit rows and 626 billing ledger lines for the window, each with decision, tier, provider, token, cost, latency, and quality fields, so the entire workload can be replayed and re-scored offline. The online learning path auto-harvested traffic during the run and produced a candidate version (v10) with 457 training samples and a validation accuracy of 0.7174; an explicit retraining command produced a further candidate (v12) with 557 samples. Both candidates were submitted to the evaluation gate, and both failed it on quality retention and cost-saving floors, so neither was promoted. The value of reporting these internal failures is that they document the loop closing: traffic becomes candidates, candidates become evidence, and evidence decides.

A latency decomposition completes the operational picture. The routing decision itself is fast: across the window-600 audit records, decision latency (the full gateway decision chain, excluding upstream generation) was approximately 19.8 ms at p50, 131 ms at p95, and 168.6 ms at p99, against a gate threshold of 2.0 seconds; end-to-end wall-clock latency including streaming generation was 9.8 s at p50 and up to 113.4 s at p99, driven by long reasoning and code outputs up to the 2,048-token ceiling. The decision layer therefore contributes a negligible share of the request latency, which is the property that makes a gate on decision latency meaningful: a candidate cannot buy gate headroom by adding decision overhead. Component-level decision latency in microbenchmarks is far lower still (P95 well under 5 ms), so the observed p99 is dominated by integration effects rather than by the learned classifier itself.

5.4 The Release Gate in Action

The most direct evidence for evaluation-driven release governance comes from the A/B adjudication of candidate versions v12 and v13 against the incumbent v7. Both candidates were generated by retraining on aggregated traffic windows and both had high decision accuracy (92.6% and 91.7%, respectively, against an expected-tier baseline of roughly 70%). Nevertheless, the blind comparison on disagreement samples showed that both candidates systematically upgraded requests that v7 had placed in the light tier, and the real generated answers on those upgraded requests did not improve: v13 retained usable quality on 18 of 29 upgrades (62.1%) versus v7's 24 of 29 (82.8%), and v12 retained only 9 of 29 (31.0%) (Figure 3). The quality-downgrade evidence triggered the methodology's manual block. The live A/B test then measured v13's absolute quality rate at 82.1% (judge n = 28) against v7's 69.7% (judge n = 66); under the then-current absolute gate of 95% the candidate failed outright, and under the relative-offset criterion adopted on 2026-10-11 (candidate quality at least control minus one point) it was marginally passable on the A/B statistics alone. The adjudication did not rely on the aggregate statistics either way: the blind review had already shown that the candidate's upgrades systematically degraded real generation quality (a measured −20.7-point drop on upgrade retention), which is the failure mode the methodology blocks regardless of the A/B numbers. The verdict was to keep v7 online.

For completeness, the incumbent v7 was itself promoted in October 2026 before the gate was enforced (a historical skip-gate release, recorded as such in the version register); it was subsequently re-verified against the gate criteria on live traffic, passing all four criteria (calibrated quality retention 0.9857, cost saving 58.09%, decision p99 21.3 ms, judge sample 70). We disclose this history because the audit trail is part of the governance claim: the gate has retrospective authority over the incumbent it protects, but it does not retroactively legitimize the original promotion.

The rollback path is not hypothetical: an earlier candidate version (v4) had been promoted and was rolled back when a smoke-regression check exposed a difficulty-underestimation defect (hard queries routed to the light tier); the rollback restored the previous incumbent from a versioned backup, as recorded in the closed-loop decision log.

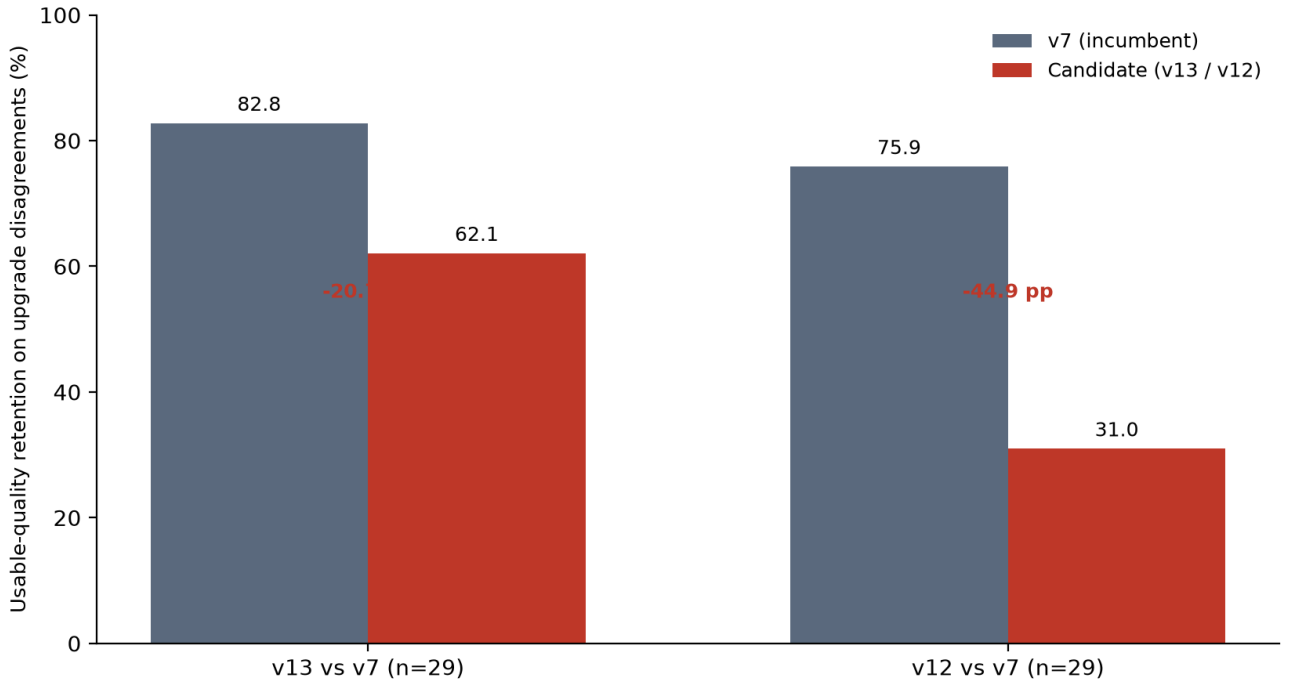


Figure 3. Blind review on upgrade disagreements: candidate versions v13 and v12 systematically degraded generated-text quality relative to the incumbent v7 (usable-quality retention 62.1% vs 82.8% for v13's window; 31.0% vs 75.9% for v12's window).

Two lessons follow. First, the candidates' high decision accuracy was misleading: it reflected agreement with a biased expected-tier label, not real generation quality, and the blind generation review caught what the label accuracy could not. The upgrade bias was quantified: among the disagreement samples, upgrades accounted for 94% of the cases (29 of 31), and the candidates' high accuracy was driven by fitting a label set that was itself biased toward escalation. Second, the gate's behavior is the system's intended failure mode. A release gate that never blocks would be useless; a gate that blocks a candidate and documents the reason is the mechanism working. The A/B split itself was verified with probe requests before the measurement window, confirming that the candidate received approximately its configured 25% share, and the candidate's upgraded behavior was observed directly in the live arm. This is the central observation we want industry practitioners to take from the system: the cost of a false promotion is paid by every subsequent request, while the cost of a false block is only a retrain cycle, so the gate should err on the side of blocking.

5.5 Judge Calibration Findings

The judge used in the production loop was itself measured. Across two independent experiments, the judge agreed with hard labels on 93.3% of samples but exhibited a systematic +13.3-point over-estimation on the difficult segment while showing zero bias on the easy segment (Figure 2). An online monitoring stream keyed to difficulty ordering triggered consecutive alerts exactly at the boundary of the difficult segment, confirming that the drift was real and localized rather than random. This finding motivates per-difficulty judge calibration, which the system now applies, and it qualifies the use of LLM-based judges in routing evaluation: a global threshold will systematically miss upgrade needs on hard queries. The practical implication is that judge bias must be measured with respect to difficulty, not only on average, because the average hides exactly the failure mode that matters most for routing decisions: the difficulty segment where upgrades are needed is the segment where the judge is least trustworthy.

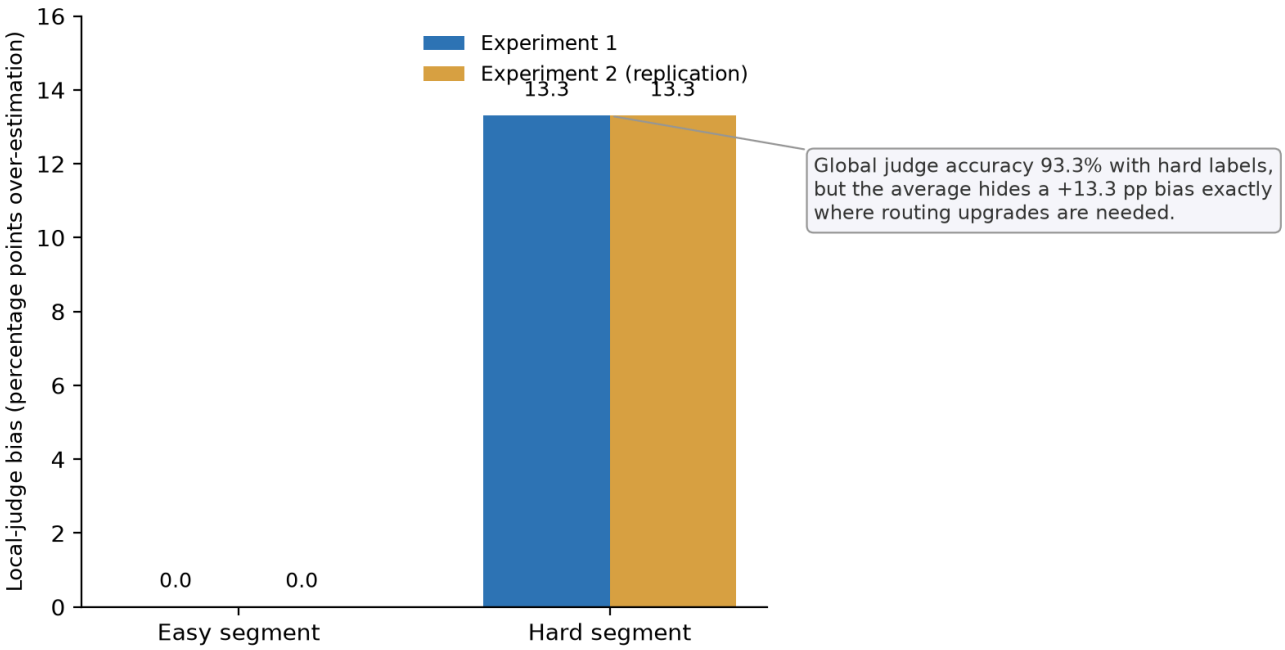


Figure 2. Local-judge bias by difficulty segment, replicated across two independent experiments: zero bias on the easy segment and a +13.3-point over-estimation on the difficult segment, which a 93.3% global agreement rate hides.

5.6 Ablation of the Governance Stages

A governance loop is only as convincing as its least replaceable component. We therefore ask, for each stage of the gate, which candidate versions it alone stopped, and which would have been released had the stage been absent. Table 5 summarizes this counterfactual analysis over the version history; the counterfactuals are inferences from the evidence that was actually collected, not new experiments.

Table 5

Ablation of governance stages: candidates each stage blocked, and the counterfactual outcome if the stage were absent

Stage removed (counterfactual)	Candidates that would have been released	Blocking evidence actually used	Outcome with the stage in place
Decision accuracy only (no blind review, no A/B statistics)	v12 (decision accuracy 92.6%) and v13 (91.7%) versus an expected-tier baseline of roughly 70%	Blind review: upgrade retention 31.0% vs 75.9% (v12, -44.9pp) and 62.1% vs 82.8% (v13, -20.7pp)	Both blocked; v7 stayed online
Aggregate A/B statistics only, relative-offset criteria (no blind review)	v13 (A/B verdict pass: quality 82.1% ≥ 69.7% - 1pp; saving 50.8% ≥ 47.2% - 2pp)	Blind review on 29 upgrade disagreements: generated answers did not improve	v13 blocked by the methodology's manual-block rule
Blind review only (no A/B statistics or cost comparison)	v2 (A/B quality 93.02% vs control 100%), v3 (93.4% vs 97.3%), v14 (cost +124.6%; saving 22.3% vs 65.4%)	A/B statistics and offline cost comparison	v2 and v3 rejected; v14 rejected
No smoke regression or rollback	v4 would have stayed online after promotion	Smoke check: hard queries routed to Light; rollback to v6	v4 rolled back before serving the defect

The four rows are mutually informative. Rows 1 and 2 show that the two most expensive stages, blind text review and live A/B testing, each blocked candidates that the other stage alone could not: aggregate A/B statistics would have passed v13 under the relative criteria adopted on 2026-10-11, and decision accuracy alone would have passed both v12 and v13. Rows 3 and 4 make the complementary point: blind review alone would not have stopped v2, v3, or v14, whose failure modes were aggregate quality rate and cost

rather than direction-specific text degradation; those were caught by the statistical and cost comparisons. The regression-and-rollback stage is the only one that operates after promotion, and v4 is the only case in which a defect reached live traffic and was reversed. In this record every stage has at least one exclusive catch, which supports the design decision to keep the loop composite rather than to rely on any single signal.

5.7 Judge-Bias Sensitivity and the Value of Calibration

Section 5.5 established that the local judge over-estimates the difficult segment by +13.3 points across two replications while showing zero bias on the easy segment, a discrepancy hidden by the 93.3% global agreement rate. Here we quantify what this means for release decisions under the absolute and relative gate criteria. Because the per-question difficulty mixture of the production workload is recorded but not itemized in this paper, we report a sensitivity analysis: Table 6 shows the systematic upward shift in the global judge score for three plausible difficult-segment shares.

Table 6

Sensitivity of the uncalibrated global judge score to the difficult-segment share p (difficult-segment bias +13.3pp, easy-segment bias opp)

Difficult-segment share p	Implied global upward bias (pp)	Consequence under the 0.95 absolute gate
10%	+1.33	Control quality (69.7%) remains far below the floor: the absolute gate is unreachable on real traffic
20%	+2.66	Same; any candidate near the floor is spuriously inflated by judge bias alone
30%	+3.99	Same; the gate's margin is distorted by the uncalibrated scale

Three observations follow. First, the global average hides exactly the segment that matters for routing: among the v12/v13 disagreement samples, 94% of the cases (29 of 31) were upgrades, and upgrades are concentrated on the harder queries, the segment where the judge is least trustworthy. A judge that is unbiased on average can therefore certify an aggressive candidate with high apparent confidence. Second, the unreachability of the absolute 0.95 floor on real traffic (control at 69.7%) was itself a symptom of an uncalibrated global scale rather than a defect of the gate: the floor became meaningful only after per-difficulty calibration (which reduced the average absolute deviation from 0.1807 to 0.1612 and drove the per-bin bias to zero) and after the authoritative re-measurement of the incumbent at 92.78% (Section 3.3). Third, the calibrated estimate is externally consistent: the local calibrated retention of 0.9278 agrees in direction with the independent external judge's 90.0% versus 81.7% result, which would be unlikely if the calibration had merely re-fit noise. We state explicitly that the sensitivity numbers in Table 6 are illustrative arithmetic over an assumed share p; they are reported to make the magnitude of the bias legible, not as a measured distribution.

5.8 Follow-up Release Windows and Release History

The v12/v13 episode (Section 5.4) was not a one-off. After the production workload, the system continued to retrain and gate candidates for another eight days. Figure 4 shows the full registered version history (v1-v17; no v11 record was generated), and Table 7 lists the gate decisions and their recorded reasons.

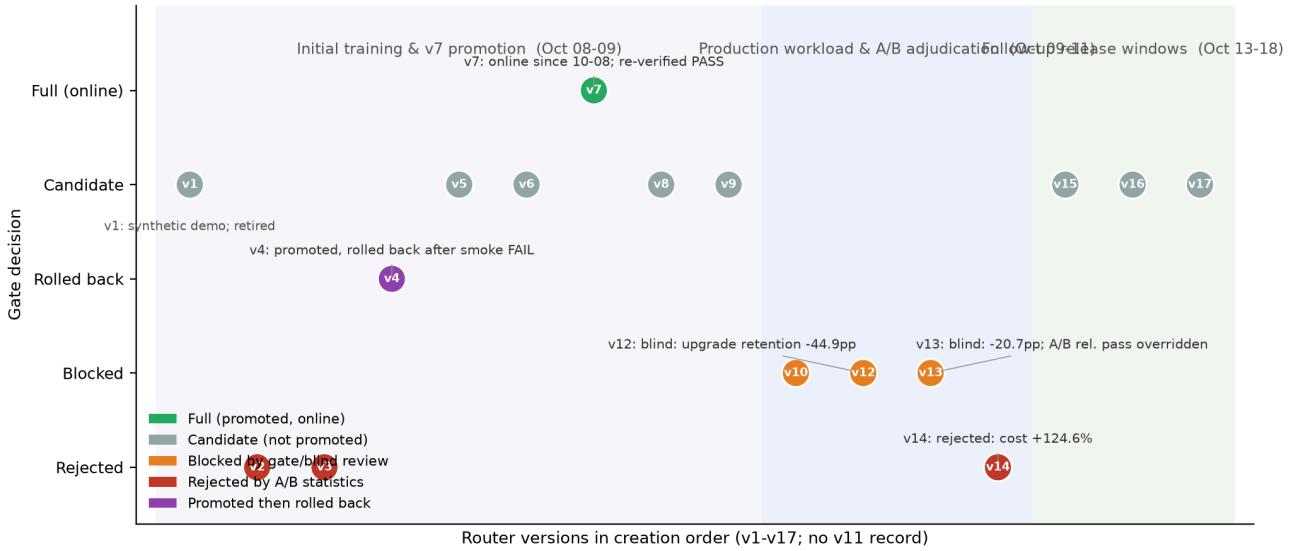


Figure 4. Release-gate decisions across the registered version history (v1-v17). Green: promoted and online; grey: candidate not promoted; orange: blocked by the gate or blind review; red: rejected by A/B statistics; purple: promoted then rolled back. Background bands mark the three release windows.

Table 7

Registered versions and release-gate decisions (2026-10-08 to 2026-10-18)

Version	Date	Samples	Val. acc.	Decision	Recorded reason
v1	10-08	106	0.9545	retired	Synthetic demo; gate skipped
v2	10-08	385	0.4026	rejected	A/B quality 93.02% vs control 100% (-6.98pp)
v3	10-08	681	1.000	rejected	A/B quality 93.4% vs 97.3% (-3.9pp)
v4	10-08	1190	0.9916	rolled back	Smoke: hard queries routed to Light
v5	10-08	571	0.9739	candidate	—
v6	10-08	619	0.9274	candidate	Predecessor of v7
v7	10-08	619	0.9274	full	Historical skip-gate; re-verified PASS post hoc
v8	10-09	779	0.9808	candidate	—
v9	10-09	1120	0.9732	candidate	—
v10	10-09	457	0.7174	blocked	Gate floors (quality retention, cost saving)
v12	10-11	557	—	blocked	Blind review: upgrade retention -44.9pp
v13	10-11	—	—	blocked	Blind review: -20.7pp; A/B relative pass overridden
v14	10-13	64	1.000	rejected	Cost +124.6%; saving 22.3% vs 65.4%
v15	10-14	262	—	candidate	Decision accuracy 17.4% vs 78.7%; relative criteria not met
v16	10-18	277	0.625	candidate	Awaiting gate
v17	10-18	277	0.625	candidate	Awaiting gate

Note. v10 and v12 were generated during and after the production workload (Section 5.3); v14-v17 constitute the follow-up release windows after the v12/v13 adjudication. Val. acc. = validation accuracy on the training/aggregation set. Dash = not recorded.

Across the sixteen registered versions, eight candidates (v2, v3, v4, v10, v12, v13, v14, v15) were subjected to the gate. All eight failed: v2, v3, and v14 on A/B statistics or cost; v10 on the offline gate floors; v12 and v13 on blind review; v4 on the regression smoke check after promotion (promoted, then rolled back); and v15 on the relative criteria (left unpromoted). Zero erroneous promotions were recorded, and the only

promoted version (v7) predates the gate and was re-verified against it post hoc (calibrated retention 0.9857, cost saving 58.09%, decision p99 21.3 ms, judged sample 70). We caution that this is a descriptive record, not a statistical characterization: with eight gated candidates from a single deployment, the false-block and false-promotion rates cannot be estimated with any precision, and the absence of recorded erroneous promotions does not establish that the gate never errs. The value of the record is that it documents the loop closing repeatedly: retraining produced candidates, the gate produced decisions, and every decision is traceable to a specific evidence file in the Aegis ledger.

Statistical resolution of the adjudication. The sample sizes in this paper were not chosen arbitrarily, and it is worth stating what they can and cannot resolve. Table 8 reports the minimal detectable difference (MDD) for a one-sided binomial test at alpha = 0.05 and power = 0.80, computed with the exact (unconditional) binomial distribution rather than a normal approximation, for the sample sizes and baseline rates that occur in the production loop. Two consequences follow. First, the blind-review windows (n = 29) resolve drops of roughly 20-23 percentage points (19.7 pp on the v13 window, 23.4 pp on the v12 window): the observed v13 drop of -20.7 pp sits at the edge of the detectable range (exact power 0.83 at that point), and the v12 drop of -44.9 pp is far beyond it (power > 0.99), so the blind stage is powered for the direction and magnitude of downgrade it is meant to catch, while remaining explicitly unable to certify small effects. Second, the A/B and production stages cover the smaller-effect regime: judge samples of n = 66 and n = 374 resolve drops of about 15.0 pp and 5.5 pp respectively, which is the reason the protocol keeps a composite of stages instead of trusting any single one. As a boundary calibration, the production retention window (n = 313, baseline 0.9278) detects a 3-point degradation with only about 60% power, so the paper never claims to certify sub-3-point retention shifts. The MDD computation is a design analysis, not a post-hoc fit; we report it so that reviewers can see how the sample sizes bound the claims, and we flag that n = 29 cannot certify effects below about 20 pp, which is why the paper never treats the blind review as a null-certifying instrument.

Table 8

Minimal detectable difference (MDD) at alpha = 0.05, power = 0.80, one-sided binomial test

Stage and baseline rate p0	n	MDD (pp)	Observed effect in this paper
Blind review, v12 window (upgrade retention 0.759)	29	23.4	-44.9 pp (31.0 vs 75.9)
Blind review, v13 window (upgrade retention 0.828)	29	19.7	-20.7 pp (62.1 vs 82.8)
Live A/B judge, candidate arm (0.697)	28	25.8	direction adjudicated with blind review
Live A/B judge, control arm (0.697)	66	15.0	control quality 69.7%
Production window judge (0.799)	374	5.5	quality retention 0.9278
Workload retention CI (0.9278)	313	4.0	95% CI [0.892, 0.953]

Note.

MDD is the smallest one-sided drop from p0 that the exact binomial test would detect with probability at least 0.80. Computation: exact (unconditional) binomial power; all values are reproducible in one scipy command (see docs/REPRODUCIBILITY.md in the companion repository).

6. Discussion

6.1 What the Results Mean

The benchmark results establish two reliable effects. First, the learned router delivers a real quality gain over a lightweight baseline on most providers, with the strongest effect on the provider whose light tier is

weakest (stepfun, +15.2 points). Second, the cost saving relative to a heavyweight baseline is consistent and large (36-51% in the benchmark, 59.65% in the production workload). Together these effects justify the economic thesis of routing: most queries do not need the flagship model, and a cheap router can find them. The magnitude of the saving is worth emphasizing because it is measured, not modeled: every number in Table 2 and Table 4 is derived from real API usage fields and real billing records, and the benchmark numbers were independently re-confirmed on a second run with a larger token budget.

The rule-tuning negative result is the more interesting finding for the research community. It shows that on this benchmark, and within this classifier family and rule grid, a well-tuned heuristic captures most of the routable signal, and that our learned router's advantage is not accuracy but calibration-free operation and cost control. This does not contradict the positive results; it refines them. Learned routing matters most when the environment is dynamic, when per-provider thresholds are unknown, and when the signal is too subtle for simple length and density heuristics. The dual-engine design reflects this: tuned rules remain the first line, and the learned router covers the long tail. We consider this negative result a contribution in its own right, because the literature overwhelmingly reports routing gains without reporting how much of the gain a careful baseline would have captured; our result is a concrete bound on that gap, and we encourage follow-up work to test whether stronger learned components (e.g., neural rankers or preference-tuned models) close it.

6.2 The Governance Loop as a Contribution

We believe the governance loop is the system's most transferable contribution. A router is not a static artifact; it is a continuously updated service, and every update is a small deployment decision that can degrade quality in ways a single metric cannot see. The loop we describe makes that decision explicit, evidence-based, and reversible. The preregistered criteria, the paired statistical tests, the blind disagreement review, and the A/B split form a protocol that can be applied to any router, and the audit trail makes every conclusion recomputable from raw evidence. The A/B episode with v12 and v13 demonstrates the value concretely: both candidates looked excellent on decision accuracy, and both were stopped by blind review of the text they actually generated. Without the governance loop, either candidate would plausibly have been promoted on the strength of its decision accuracy, and the deployment would have paid more for answers that were no better.

For industry-track and systems venues, this protocol-level contribution may be more valuable than any single algorithmic improvement, because it addresses the failure mode that actually occurs in production: silent regression. The systems community has developed mature practices for rolling out models and services; this paper shows those practices, adapted to the specifics of routing (a learned component with a quality/cost trade-off and an LLM judge), working on real traffic.

6.3 Limitations and Threats to Validity

We organize the threats to validity by the standard four-way taxonomy; no new evidence appears in this section, but the structure makes the boundaries of the claims explicit.

Construct validity. The judge is a single local model, and although we measure and disclose its difficulty-dependent bias, the quality numbers inherit that bias; the reported confidence interval for the production quality retention reflects sampling error only and not judge bias, which we address separately through calibration and monitoring. The external blind evaluation provides a cross-check but uses a different judge, a different question set, and different sampling, so the two quality figures should not be

averaged or directly compared; after the final calibration run, they are consistent in direction (the routing arm exceeds the flagship control on the external judge, and the local calibrated estimate lies in the same region). Decision accuracy in Section 5.4 uses expected-tier labels that are themselves biased toward escalation, which is why the gate never relies on label accuracy alone.

Internal validity. Comparisons are paired and preregistered (Section 4.2), which prevents the protocol from being redefined in light of the results; the remaining internal threat is the modest effective sample size per arm after truncation exclusion, whose consequences the statistical-resolution analysis in Section 5.8 quantifies rather than hand-waves.

External validity. The benchmark question set is drawn from public math and multiple-choice sources, so the results may not transfer to open-ended or conversational workloads; the production workload, while paid and real, is a single 313-question window from one operator, and the routed tier distribution is heavily light-weighted (Section 5.3), so the profile is not representative of every traffic mix. The gitee provider lacks a heavy tier, so its results are partially unobservable.

Conclusion validity. The release-gate evidence comes from a single deployment and a single operator: although the version register now spans sixteen versions and eight gated candidates across three windows, it is not yet sufficient to estimate the gate's false-block and false-promotion rates statistically (Section 5.8), and no second deployment or independent operator exists to test the protocol's portability. Cross-operator replication is therefore the first priority for follow-up work, and Section 4.6 specifies the protocol for it.

6.4 Implications for Practice

For practitioners, three recommendations follow from this work. First, evaluate routers with preregistered paired tests and equivalence bounds rather than point estimates, and report the conservative caliber alongside the fair one. Second, calibrate LLM judges per difficulty segment and monitor them online; a judge with localized bias will silently corrupt routing decisions if left uncalibrated. Third, treat router releases as governed deployments: version the router strategy, gate candidates on evidence, blind-review disagreements before trusting label accuracy, and keep the audit trail that makes the whole loop reproducible. None of these recommendations requires a particular routing algorithm; they are compatible with any router that can be versioned and run against a control, which is to say any router that can be deployed at all.

6.5 Relation to Prior Internal Reporting

The figures reported in this paper come from the preregistered evaluation campaign described in Section 4. They are the campaign's window-specific measurements (36-51% benchmark saving, 59.65% production saving, 0.9278 quality retention). Our product's externally disclosed performance snapshot, published separately for commercial purposes (33.92% cost saving, 0.8865 quality retention, 8.65 ms decision p99, over a different audit window with 384 audit rows and 185 judged samples), reflects a later window and a different disclosure baseline. Both sets of numbers are traceable to the same audit and billing ledger and can be recomputed from it; they are not interchangeable, and readers should not average across windows. We state this explicitly because a governance claim depends on the discipline of reporting exactly which window and which protocol a number came from.

7. Conclusion

This paper described a production system for LLM routing in which deployment is governed by an evidence-based release loop rather than by point estimates or intuition. The system combines a four-stage routing gateway with a six-loop governance cycle spanning observation, aggregation, retraining, selection, A/B testing, and adjudication. Our evaluation on a 2,000-question, five-provider benchmark and a 313-question paid production workload showed statistically significant quality gains over a lightweight baseline on three of four providers, cost savings of 36-51% in the benchmark and 59.65% in production, a 98.1% success rate, calibrated quality retention of 0.9278 (95% CI [0.892, 0.953]), and a decision-latency p99 of 0.169 seconds. It also produced a practically important negative result: after rule tuning, our learned router did not significantly outperform the best-tuned heuristic on any provider of this benchmark, reframing the value of learned routing as calibration-free operation and cost control within the evaluated scope. Finally, the release gate blocked two candidate versions that systematically downgraded quality on upgraded requests, keeping the incumbent online and demonstrating the intended failure mode of a working governance mechanism.

The central claim of this paper is that evaluation-driven release governance, not the routing algorithm alone, is what makes LLM routing safe and economical in production. The protocol, the audit trail, the disclosed threshold history, and the documented failure mode are the transferable parts of this work. Future work should extend the evaluation to open-ended workloads, accumulate a broader base of blocked and promoted candidates to characterize the gate statistically, calibrate judges jointly across difficulty and domain, test stronger learned components against the tuned-rule bound reported here. It should also deploy a public routing baseline (e.g., RouteLLM with its official weights) on the same bench2000 questions under the same judge and cost model, so that the algorithm-independent governance protocol can be compared with a leading learned router on identical ground truth, and publish the full artifact bundle (benchmark manifest, re-check tooling, and audit evidence) so that the governance protocol itself can be tested by other operators on their own traffic. A second deployment, ideally at an independent operator, remains the single most informative next experiment: it would test whether the protocol's stage-level behavior (Section 5.6) and threshold-revision discipline (Section 3.3) transfer beyond this one environment.

Acknowledgments

The author thanks the maintainers of the open-source components used in the evaluation pipeline. This manuscript was drafted with the assistance of AI writing tools; all experimental claims, figures, and statistical results were verified by the author against the raw audit records, evaluation logs, and re-check artifacts before submission. The author is an undergraduate student at Xi'an University of Technology; this work was conducted independently as a production engineering and research project.

References

Bao, Z. (2026). OrcaRouter: A production-oriented LLM router with hybrid offline-online learning. arXiv. https://doi.org/10.48550/arXiv.2605.30736

Chen, H., Liu, X., He, B., Lyu, F., Chen, Y., Liu, X., Liu, Y., & Jiang, J. (2026). The workload-router-pool architecture for LLM inference optimization: A vision paper from the vLLM Semantic Router project. arXiv. https://doi.org/10.48550/arXiv.2603.21354

Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv. https://doi.org/10.48550/arXiv.2305.05176

Dekoninck, J., Baader, M., & Vechev, M. (2025). A unified approach to routing and cascading for LLMs. In Proceedings of the 42nd International Conference on Machine Learning (ICML 2025). https://doi.org/10.48550/arXiv.2410.10347

Feng, A., Srinivasan, B., Zhou, Y., Xu, Z., Zhou, K., Guan, S., Xu, Z., Tian, H., Ding, H., Sun, C., Cheong, L. L., Ding, B., et al. (2025). IPR: Intelligent prompt routing with user-controlled quality-cost trade-offs. arXiv. https://doi.org/10.48550/arXiv.2509.06274

Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, H., Liu, S., Ma, X., et al. (2024). A survey on LLM-as-a-judge. arXiv. https://doi.org/10.48550/arXiv.2411.15594

Huang, Z., Ling, G., Lin, Y., Chen, Y., Zhong, S., Wu, H., & Lin, L. (2025). RouterEval: A comprehensive benchmark for routing LLMs to explore model-level scaling up in LLMs. In Findings of the Association for Computational Linguistics: EMNLP 2025 (pp. 3860-3887). https://doi.org/10.18653/v1/2025.findings-emnlp.208

Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Ke, H., et al. (2024). RouterBench: A benchmark for multi-LLM routing system. arXiv. https://doi.org/10.48550/arXiv.2403.12031

Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, Z., Moens, T., Kim, D., Joshi, J., et al. (2025). Universal model routing for efficient LLM inference. arXiv. https://doi.org/10.48550/arXiv.2502.08773

Jung, J., Brahman, F., & Choi, Y. (2025). Trust or escalate: LLM judges with provable guarantees for human agreement. In Proceedings of the International Conference on Learning Representations (ICLR 2025). https://proceedings.iclr.cc/paper_files/paper/2025/file/08dabd5345b37fffcbe335bd578b15a0-Paper-Conference.pdf

Kohl, J., Gloger, L., Costa, R., Kruse, O., Luitz, M. P., Katz, D., Mukherjee, R., Rajagopalan, B., et al. (2024). Generative AI toolkit: A framework for increasing the quality of LLM-based applications over their whole life cycle. arXiv. https://doi.org/10.48550/arXiv.2412.14215

Krumdick, M., Lovering, C., Reddy, V., Ebner, S., & Tanner, C. (2025). No free labels: Limitations of LLM-as-a-judge without human grounding. arXiv. https://doi.org/10.48550/arXiv.2503.05061

Liu, Y. (2023). G-Eval: NLG evaluation using GPT-4 with better human alignment. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. https://doi.org/10.18653/v1/2023.emnlp-main.153

Lu, Y., Liu, R., Yuan, J., Cui, X., Zhang, S., Liu, H., & Xing, J. (2026). RouterArena: An open platform for comprehensive comparison of LLM routers. In Proceedings of the International Conference on Learning Representations (ICLR 2026). https://doi.org/10.48550/arXiv.2510.00202

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv. https://doi.org/10.48550/arXiv.2406.18665

Panda, P., Magazine, R., Devaguptapu, C., Takemori, S., & Sharma, V. (2025). Adaptive LLM routing under budget constraints. In Findings of the Association for Computational Linguistics: EMNLP 2025. https://doi.org/10.18653/v1/2025.findings-emnlp.1301

Somerstep, S., Polo, F. M., de Oliveira, A. F. M., Mangal, P., Silva, M., Bhardwaj, O., Yurochkin, M., & Maity, S. (2025). CARROT: A cost aware rate optimal router. arXiv. https://doi.org/10.48550/arXiv.2502.03261

Srivatsa, K. A., Maurya, K. K., & Kochmar, E. (2024). Harnessing the power of multiple minds: Lessons learned from LLM routing. In Proceedings of the Fifth Workshop on Insights from Negative Results in NLP. https://doi.org/10.18653/v1/2024.insights-1.15

Tripathi, T., Wadhwa, M., Durrett, G., & Niekum, S. (2025). Pairwise or pointwise? Evaluating feedback protocols for bias in LLM-based evaluation. In Proceedings of the First Conference on Language Modeling (COLM 2025). https://doi.org/10.48550/arXiv.2504.14716

Wang, X., Liu, Y., Cheng, W., Zhao, X., Chen, Z., Yu, W., Fu, Y., & Chen, H. (2025). MixLLM: Dynamic routing in mixed large language models. In Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2025). https://doi.org/10.48550/arXiv.2502.18482

Wataoka, K., Takahashi, T., & Ri, R. (2024). Self-preference bias in LLM-as-a-judge. arXiv. https://doi.org/10.48550/arXiv.2410.21819

Wu, F., & Silwal, S. (2025). Efficient training-free online routing for high-volume multi-LLM serving. In Advances in Neural Information Processing Systems 38 (NeurIPS 2025). https://papers.nips.cc/paper_files/paper/2025/file/c878e34a1b2095ae8177961ba24d642d-Paper-Conference.pdf

Zhang, Y., Xie, F., Yang, S., & Wu, C. (2025). Meta-router: Bridging gold-standard and preference-based evaluations in large language model routing. arXiv. https://doi.org/10.48550/arXiv.2509.25535

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and chatbot arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023).

Zhu, L., Wang, X., & Wang, X. (2025). JudgeLM: Fine-tuned large language models are scalable judges. In Proceedings of the International Conference on Learning Representations (ICLR 2025). https://proceedings.iclr.cc/paper_files/paper/2025/file/7f8f73134e253845a8f82983219a8452-Paper-Conference.pdf